Selectors and style rules

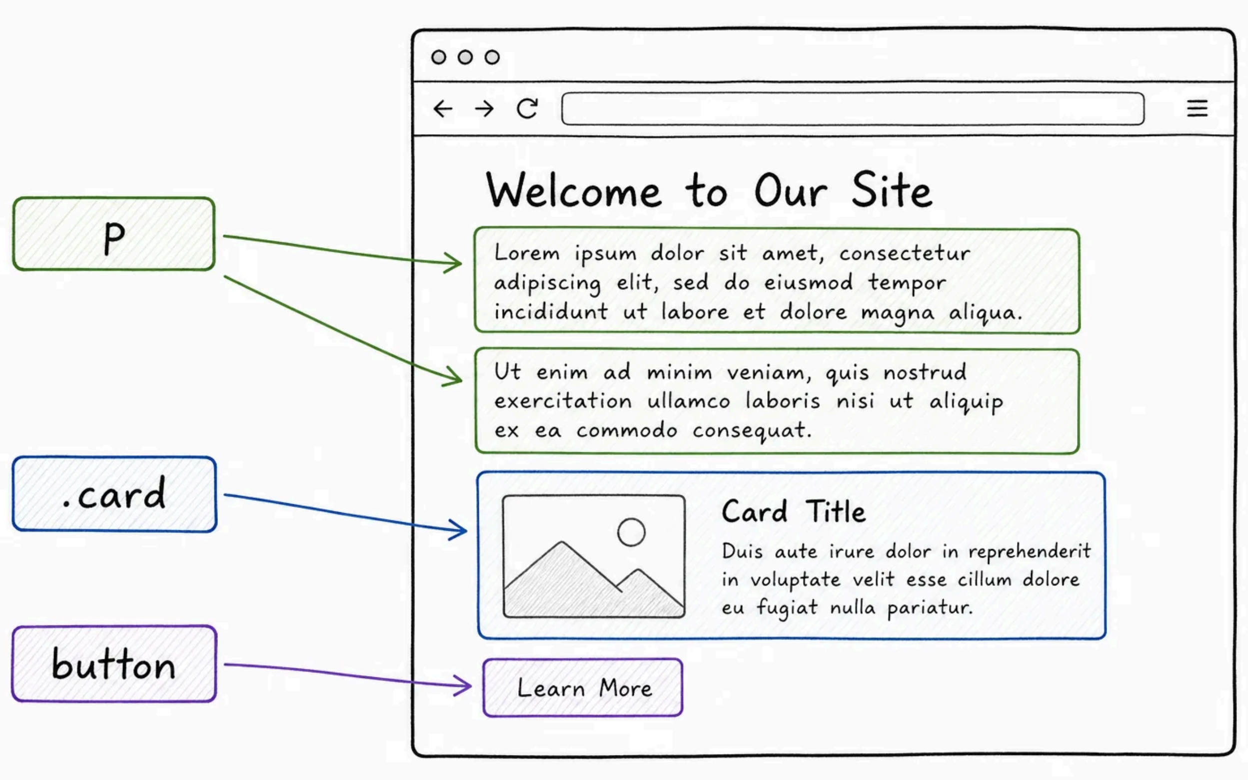
In the previous lesson, you saw that CSS is made of style rules. A rule has a selector, then a set of declarations inside curly braces.

css

```css
p {
  color: #444;
  font-size: 18px;
}
```

This lesson focuses on the selector part. A selector tells the browser which elements a rule should apply to.

Think of a selector as a way to ask a question about the DOM: "Find every paragraph", "find every element with this class", or "find every link inside this navigation area." The browser checks the page against that selector and applies the declarations to every element that matches.



Selectors choose elements. Declarations style the elements that match.

Element selectors

An element selector targets every element with a given tag name.

css

```css
p {
  line-height: 1.6;
}
```

That rule applies to every `<p>` element on the page.

Element selectors are useful for broad page defaults:

css

```css
body {
  font-family: system-ui, sans-serif;
}

h1 {
  font-size: 40px;
}

a {
  color: #047857;
}
```

These rules are saying, "all body text should use this font", "all top-level headings should start at this size", and "all links should use this color."

That is a good use of element selectors. They work well when you really mean every element of that type, or when you are setting a default that more specific component styles can build on later.

They are less useful when you only want to style one particular part of the page. If you style every `button` on the page, then every button gets that style: submit buttons, menu buttons, delete buttons, modal buttons, everything. That is often too broad.

Class selectors

A class selector targets elements by their `class` attribute. In HTML, the class is written without a dot:

html

Previous Lesson Next Lesson

```html
</article>
```

In CSS, you select that class with a dot:

css

```css
.card {
  padding: 24px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}
```

The dot means "look for this class name."

Classes are the main styling hook you will use in CSS. They are reusable, readable, and not tied to one specific HTML tag. A `.card` class can go on an `article`, a `section`, a `div`, or any other element that should look like a card.

html

```html
<article class="card">Starter plan</article>
<section class="card">Team plan</section>
<div class="card">Enterprise plan</div>
```

Because the class name is the same, all three elements match `.card`.

The dot only belongs in CSS - Write `class="card"` in HTML, but write `.card` in CSS. Beginners often accidentally write `class=".card"` in HTML. The dot is not part of the class name. It is part of the CSS selector.

Multiple classes on one element

One element can have more than one class. In HTML, separate class names with spaces:

html

```html
<button class="button primary-button">Save changes</button>
<button class="button secondary-button">Cancel</button>
```

This lets you split shared styles from variant styles:

css

```css
.button {
  padding: 12px 16px;
  border: 0;
  border-radius: 6px;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}

.secondary-button {
  background-color: #e5e7eb;
  color: #111827;
}
```

Both buttons share the `.button` styles. Each button also gets its own variant style.

This pattern shows up everywhere in real interfaces. A card might be both `.card` and `.featured-card`. A message might be both `.alert` and `.error-alert`. A navigation link might be both `.nav-link` and `.active`.

The point is not to create a class for every single property. The point is to give the element meaningful styling hooks.

Combining selectors

You can combine selector parts with no space between them. This means one element must match all the parts.

```css
.card.featured-card {
  background-color: #f9fafb;
}
```

That selector means "an element that has both `card` and `featured-card` in its class list."

Now compare it with a selector that has a space:

```css
.card .featured-card {
  background-color: #f9fafb;
}
```

That selector has a space, so it means "an element with class `featured-card` inside an element with class `card`." That is a different shape in the DOM.

Small spacing differences can completely change what a selector means.

> ⚠ **Spaces change selectors** - `.card.featured-card` means one element with both classes. `.card .featured-card` means an element with class `featured-card` somewhere inside an element with class `card`.

ID selectors

An ID selector targets one element by its `id` attribute. In HTML, the ID is written without a `#`:

```html
<section id="pricing">
  <h2>Pricing</h2>
</section>
```

In CSS, you select it with `#`:

```css
```

```css
#pricing {
  background-color: #f9fafb;
}
```

IDs are meant to be unique. Only one element on a page should use a given ID. That makes IDs useful for things like page anchors:

html

```html
<a href="#pricing">Jump to pricing</a>
```

When the user clicks that link, the browser jumps to the element with `id="pricing"`.

For styling, classes are usually the better default because they are reusable, while IDs are unique per page. If you style with IDs everywhere, your CSS becomes harder to reuse and harder to adjust later.

⚠ **Use IDs carefully in CSS** - IDs have their place, especially for unique page anchors and JavaScript hooks. For normal styling, prefer classes unless you truly mean one specific element on the page.

Group selectors

Sometimes several selectors should get the same declarations. Instead of writing the same rule multiple times, separate the selectors with commas:

css

```css
h1,
h2,
h3 {
  line-height: 1.2;
}
```

That rule applies to all `h1`, `h2`, and `h3` elements.

The comma matters because it means "match this selector, and also this selector, and also this selector."

Descendant selectors

A descendant selector targets elements inside another element.

css

```css
nav a {
  text-decoration: none;
}
```

This rule applies to links inside a `nav`. It does not apply to every link on the page.

That is useful when the same HTML element should look different in different places:

html

```html
<nav class="site-nav">
  <a href="/">Home</a>
  <a href="/pricing">Pricing</a>
</nav>

<main>
  <p>Read the <a href="/docs">full documentation</a>.</p>
</main>
```

css

```css
.site-nav a {
  color: #111827;
  text-decoration: none;
}

main a {
  color: #2563eb;
  text-decoration: underline;
}
```

Navigation links and content links are both `<a>` elements, but they live in different parts of the DOM. Selectors let you style them differently.

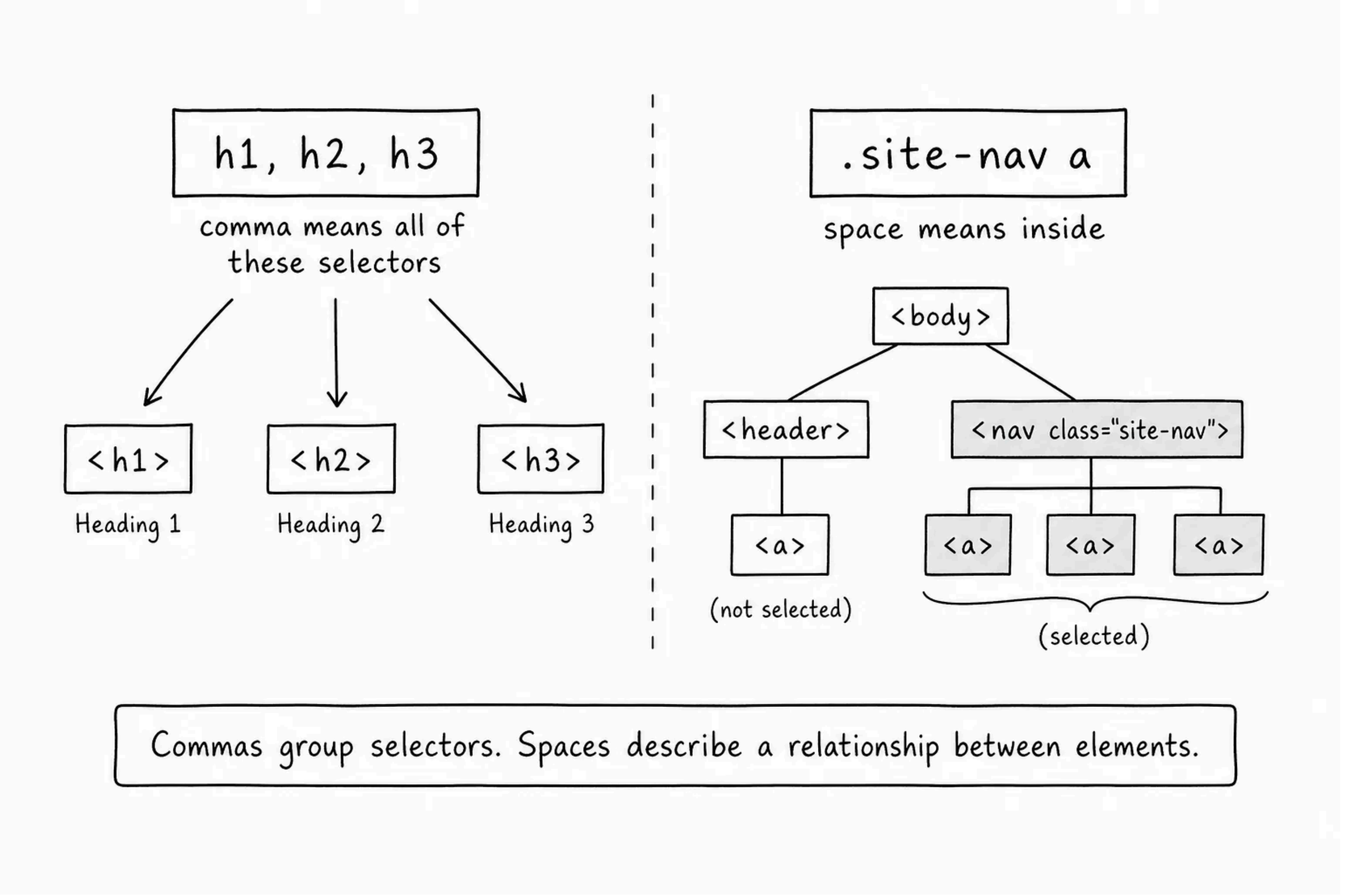
Descendant selectors are practical, but do not make them too long:

css

```css
main article section div p a {
  color: red;
}
```

That selector is hard to read and easy to break. If a selector needs to describe the whole page structure to find one thing, a class is probably clearer.

Use the nearest useful hook - If you need to style links in the main navigation, `.site-nav a` is usually clearer than `header nav ul li a`. Use the class that names the region, then select the element inside it.



Name what something is, not what it looks like

Good selectors are not just technically correct. They also make the page easier to understand.

Prefer class names that describe the role of the thing:

html

```html
<p class="error-message">Email is required.</p>
```

css

```css
.error-message {
  color: #b91c1c;
}
```

Avoid class names that only describe the current look:

html

```html
<p class="red-text">Email is required.</p>
```

The problem with `red-text` is that the name will become wrong if the design changes from red to orange, or if error messages later need an icon, border, or background. `error-message` describes what the element is in the interface, so the CSS can change without making the HTML feel outdated.

This does not mean every class name has to be perfect. It means you should try to name the thing's job, not only the first style you gave it.

When a selector does not work

When a style does not apply, the first question is simple: did the selector match anything?

When the selector does not match, check these common mistakes:

- The stylesheet is not linked correctly.
- The class name is spelled differently in HTML and CSS.
- The dot is missing from a class selector.
- The `#` is missing from an ID selector.
- A space changed the meaning of the selector.
- The element you expected is not actually in the DOM. Check the Elements tab in DevTools to confirm it is there.

DevTools helps here: inspect the element, look at its tag name, classes, and ID, then compare them with your selector. If the selector does not describe the element you clicked, the rule will not apply.

AI Tutor

Explain how to choose between an element selector, a class selector, an ID selector, and a descendant selector. Use a simple webpage with a header, navigation links, content links, and a card, but focus on the reasoning behind each selector choice.

Try it

For this selector exercise, use a small two-file page: `index.html` and `style.css`.

Start with this HTML:

html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Selector practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <header class="site-header">
      <h1>Selector practice</h1>
      <nav class="site-nav">
        <a href="/">Home</a>
        <a href="#pricing">Pricing</a>
      </nav>
    </header>

    <main>
      <p class="lead">This page uses different selectors for different jobs.</p>

      <section id="pricing" class="card featured-card">
        <h2>Starter plan</h2>
        <p>Good for small projects.</p>
        <a href="/signup" class="button primary-button">Choose plan</a>
      </section>
```

```
    <p class="error-message">This is an example error message.</p>
  </main>
</body>
</html>
```

Then add these styles:

```css
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  max-width: 720px;
  margin: 40px auto;
}

h1,
h2 {
  line-height: 1.2;
}

.site-nav a {
  color: #111827;
  text-decoration: none;
  margin-right: 12px;
}

.lead {
  font-size: 20px;
  color: #4b5563;
}

.card {
  padding: 24px;
  margin-top: 24px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}

.featured-card {
  background-color: #f9fafb;
}

.button {
  display: inline-block;
  padding: 10px 14px;
  border-radius: 6px;
  text-decoration: none;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}

.error-message {
  color: #b91c1c;
}
```

Open the page in your browser and try these changes:

- Change `.lead` to `lead` in the CSS and reload. The style stops applying because `lead` looks for a `<lead>` element, not a class.
- Change `.site-nav a` to `a` and reload. Now every link gets the navigation link style.

- Change `.card` to `#pricing` and reload. The styles still apply because both `.card` and `#pricing` happen to match the same element. But `#pricing` would not style a second card if you added one.
- Change `.featured-card` to `.card.featured-card` and reload. The style still applies because the section has both classes.
- Change `.card.featured-card` to `.card .featured-card` and reload. The background stops applying because there is no nested `.featured-card` element inside `.card`.
- Open DevTools, inspect the pricing section, and compare its tag name, ID, and classes with the selectors in your CSS.

The goal is to practice reading a selector as a sentence: `h1, h2` means all `h1` and `h2` elements, `.site-nav a` means links inside `.site-nav`, and `.card.featured-card` means one element with both classes.

0 / 12

Knew 0 Items Learnt 0 Items Skipped 0 Items ResetProgress

Test Your Understanding

What does a CSS selector do?

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →